STEVENS
INSTITUTE OF TECHNOLOGY
1870

Reinforcement Learning 1

CS 541 Lecture 9

Zining Zhu

October 29, 2026



STEVENS
INSTITUTE OF TECHNOLOGY
1870

The reinforcement learning problem

Agent · Environment · Reward · Return

From supervised learning to RL

Supervised learning learns from labeled examples; RL learns from **consequences** of actions.

- No fixed dataset of (x, y) pairs; the agent generates its own data by acting
- Feedback is a scalar **reward**, often delayed and sparse
- Actions affect future observations, not just the current prediction
- Goal: learn a **policy**, a mapping from situations to actions, that maximizes cumulative reward

What makes RL hard

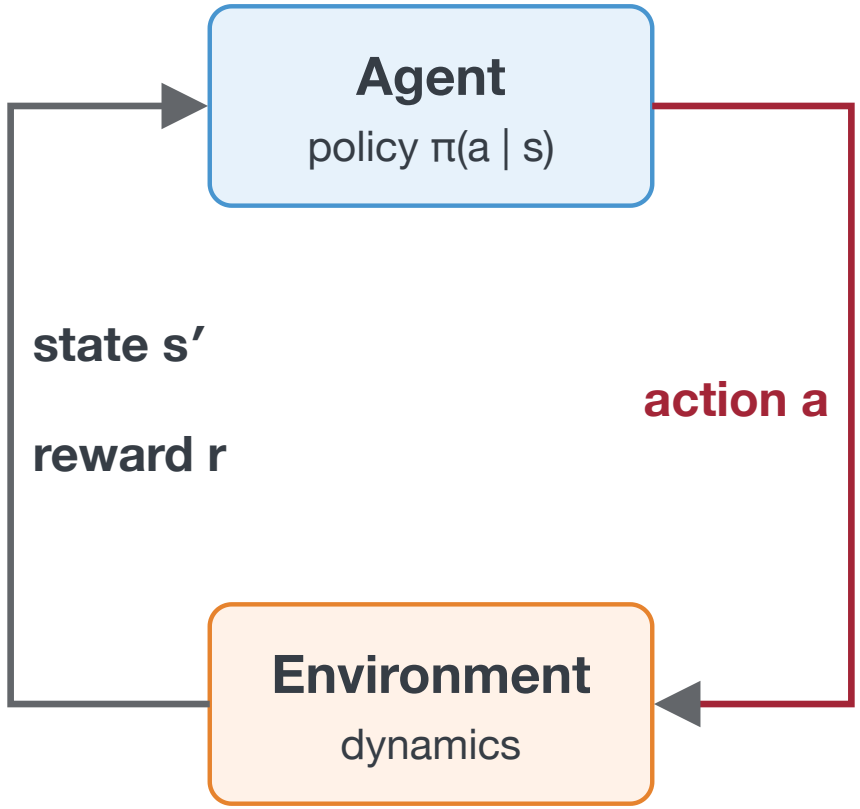
Three properties that supervised learning never had to deal with.

- **Delayed credit**: a reward may arrive many steps after the action that caused it
- **Sequential dependence**: today's action changes tomorrow's situation, not just today's label
- **The data distribution is a moving target**: as the policy improves, the states it visits change too
- No oracle ever says "the correct action here was X" — only "here is how well things went"

The agent-environment loop

At each time step, the agent and environment exchange one round of information.

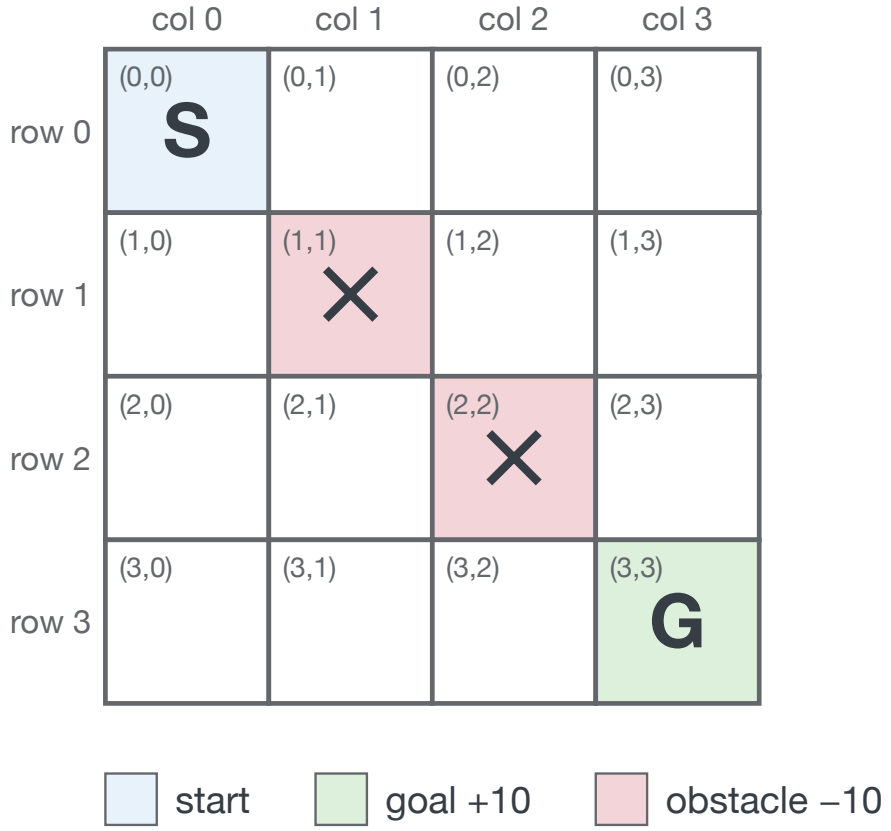
- Agent observes a **state** s
- Agent selects an **action** a according to its policy
- Environment returns a **reward** r and a **next state** s'
- Loop repeats, generating a **trajectory**: $s_0, a_0, r_0, s_1, a_1, r_1, \ldots$



A running example: GridWorld

We will use one small, complete example throughout today's lecture.

- A 4×4 grid of cells, indexed (row, col) from (0,0) at the top-left to (3,3) at the bottom-right
- The agent starts at (0,0)
- **Goal** cell at (3,3): reaching it ends the episode
- **Obstacle** cells at (1,1) and (2,2): stepping into one ends the episode with a penalty
- Every other cell is free to move through



Every worked calculation today reuses this GridWorld.

GridWorld: actions and transitions

- Four actions: **up**, **down**, **left**, **right**
- Each action moves the agent one cell in that direction
- Moving into a wall leaves the agent in place (the action is legal but has no effect)
- Transitions are **deterministic** here for simplicity: choosing “right” always moves right if possible

GridWorld: rewards

Event	Reward
Each ordinary step	**−1**
Stepping into an obstacle	**−10, episode ends**
Reaching the goal	**+10, episode ends**

The −1 per-step cost encourages the agent to reach the goal **quickly**, not just eventually — a common pattern: shape the reward so that "doing nothing forever" is not free.

GridWorld: one sample trajectory

Suppose the agent follows the path right, right, right, down, down, down (avoiding both obstacles):

- $s_0 = (0,0), a_0 = \text{right}, r_0 = -1, s_1 = (0,1)$
- $s_1 = (0,1), a_1 = \text{right}, r_1 = -1, s_2 = (0,2)$
- $s_2 = (0,2), a_2 = \text{right}, r_2 = -1, s_3 = (0,3)$
- $s_3 = (0,3), a_3 = \text{down}, r_3 = -1, s_4 = (1,3)$
- $s_4 = (1,3), a_4 = \text{down}, r_4 = -1, s_5 = (2,3)$
- $s_5 = (2,3), a_5 = \text{down}, r_5 = +10, s_6 = (3,3)$ – **goal reached, episode ends**

Markov Decision Process (MDP)

The formal object that RL problems are posed over.

- **States** S: everything relevant needed to decide the next action
- **Actions** A: choices available to the agent
- **Transition function** $P(s' \mid s, a)$: how the world responds to an action
- **Reward function** $R(s, a, s')$: the immediate payoff
- **Discount factor** $\gamma \in [0, 1)$: how much future reward is worth today
- **Markov property**: the next state depends only on the current state and action, not the full history

The Markov property, tested on GridWorld

- In GridWorld, the current (row, col) cell tells you everything needed to predict what happens next
- You do *not* need to remember how the agent got there — the path taken before is irrelevant to what "right" does now
- This is what makes (row, col) alone a valid **state**: it is a **sufficient statistic** for the future
- Contrast: if reward depended on "have you visited (1,1) before," the cell alone would no longer be Markov — you would need to augment the state with a visited-set

Policy: the agent's decision rule

- **Policy** $\pi(a \mid s)$: the agent's rule for choosing actions
- **Deterministic** policy: $a(s)$ is a single fixed action per state, e.g. "always move right in (0,0)"
- **Stochastic** policy: $\pi(a \mid s)$ is a probability distribution over actions, e.g. 70% right, 30% down
- In GridWorld, a stochastic policy might assign probabilities to each of the 4 actions per cell
- Why prefer stochastic policies during learning: they naturally support **exploration** (Section 2)

Return: scoring a whole trajectory

- **Return** G: the discounted sum of future rewards from a point in the trajectory onward
- $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$
- G_0 (the return of the whole episode) is what we ultimately want to be large
- Because rewards are scalars but trajectories are long sequences, **return** is how we compress an entire trajectory's quality into one number

Example: computing G on GridWorld

Using the 6-step trajectory above with discount $\gamma = 0.9$, computing the return from the start:

- $G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \gamma^4 r_4 + \gamma^5 r_5$
- $G_0 = (-1) + 0.9(-1) + 0.81(-1) + 0.729(-1) + 0.6561(-1) + 0.59049(+10)$
- $G_0 = -1 - 0.9 - 0.81 - 0.729 - 0.6561 + 5.9049 \approx \mathbf{1.82}$
- Even though every intermediate reward is negative, the large final reward (discounted only mildly, since it arrives soon) makes the trajectory net positive

Discount factor γ: why it exists

- γ close to **0**: the agent is myopic, caring almost only about the immediate reward
- γ close to **1**: the agent is farsighted, weighing distant future rewards almost as much as immediate ones
- $\gamma < 1$ also keeps infinite-horizon returns finite (a geometric series), which matters when episodes don't have a fixed end
- Typical values in practice: 0.9 to 0.999, chosen based on how long-term the task's consequences are

Value functions

- **Value function** $V^{\pi}(s)$: expected return starting from state s, then following policy π
- $V^{\pi}(s) = \mathbb{E}[\, G_t \mid s_t = s, \pi \,]$
- **Action-value function** $Q^{\pi}(s, a)$: expected return starting from taking action a in state s, then following π
- Relation between them: $V^{\pi}(s) = \sum_a \pi(a \mid s)\, Q^{\pi}(s, a)$ — the value of a state is the action-value averaged over what the policy actually does there

The objective of reinforcement learning

- Find the policy π^* that maximizes expected return from the start state(s)
- $\pi^* = \arg\max_\pi \mathbb{E}[\, G_0 \mid \pi \,]$
- Everything else in this unit — Bellman equations, policy gradients, actor-critic — is a strategy for approximately solving this one optimization problem
- The strategies differ in *what they estimate* (values, policies, or both) and *how they use experience* (planning with a model, or learning model-free)

Check yourself: Section 1

- Can you write down the 5-tuple that defines an MDP?
- Why is (row, col) alone a valid Markov state in GridWorld, but “current cell + whether goal ever seen” would still work while “steps taken so far” alone would not?
- Given a trajectory’s rewards and a discount factor, can you compute G_0 by hand?
- What is the difference between $V^\pi(s)$ and $Q^\pi(s, a)$, in one sentence each?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Exploration, exploitation, and two ways to learn

Model-free vs. model-based

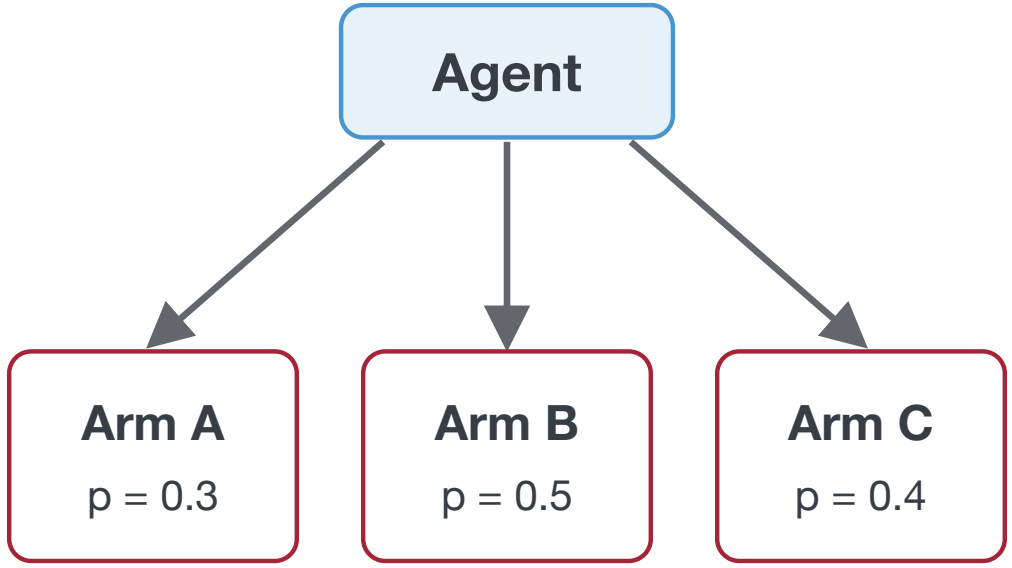
The exploration-exploitation tradeoff

An agent that only exploits what it currently believes is best can get stuck.

- **Exploitation**: take the action currently believed to be best
- **Exploration**: try a different action to learn whether it is actually better
- Committing too early to a policy risks missing a better one
- This tradeoff exists in almost every learning-from-interaction problem, not just full RL

A simpler testbed: multi-armed bandits

- Imagine 3 slot machines (“arms”), each with an unknown, fixed probability of paying out
- Arm A pays out with probability 0.3, arm B with 0.5, arm C with 0.4 (unknown to the agent)
- Each round: pick one arm, observe a reward (payout or not), repeat
- No state transitions at all — this isolates the exploration-exploitation tradeoff from everything else in RL



p is hidden from the agent. Each pull returns reward 1 (payout) or 0.

Example: bandit pulls and running averages

Suppose the agent pulls arm A five times and observes payouts 1, 0, 1, 0, 0:

- Running average estimate after each pull: 1.0, 0.5, 0.67, 0.5, 0.4
- After only 5 pulls, the estimate is noisy — the true probability is 0.3, and 0.4 is a plausible over-estimate
- If the agent locks in on the *first* arm that looks good and never tries the others, it may never discover that arm B (true rate 0.5) is actually better

ε-greedy: a concrete exploration rule

- With probability $1 - \varepsilon$: take the action currently estimated to be best (**exploit**)
- With probability ε: take a uniformly random action instead (**explore**)
- Example with $\varepsilon = 0.1$ over 100 pulls: about 90 pulls exploit the current best-looking arm, about 10 pulls sample a random arm
- Simple to implement, and a very common default in practice — but exploration is "blind": a bad-looking arm gets the same random shot as a merely unlucky one

Other exploration strategies (brief survey)

- **Optimism under uncertainty** (e.g. UCB — upper confidence bound): prefer arms/actions whose estimate is *uncertain*, not just high, so under-tried options get a bonus
- **Thompson sampling**: maintain a belief distribution over each arm's true rate, sample from it, and act on the sample
- **Entropy bonuses**: in policy-gradient methods (Section 3), add a term to the objective that rewards keeping $\pi(\cdot \mid s)$ spread out, discouraging premature certainty
- We will not derive these in depth — the point is that "add randomness to your greedy choice" is only the simplest member of a larger family

Exploration in GridWorld

- A purely greedy agent that never explores might latch onto the first path it stumbles across, even if it takes an unnecessarily long route around the obstacles
- Occasional random moves let the agent discover a shorter path it would never have tried under pure exploitation
- The cost of exploring is real: some episodes will be worse than the agent’s current best guess, purely to gather information

Model-based vs. model-free RL

Two broad strategies for turning experience into a policy.

- **Model-based**: learn (or are given) the transition function P and reward function R, then plan with them
  - Can simulate rollouts internally; sample-efficient but requires an accurate model
- **Model-free**: learn a policy or value function directly from experience, without modeling P or R
  - No model to be wrong about; typically needs more interaction data
- This course focuses on **model-free** methods: policy gradients and actor-critic

GridWorld, model-based style

- If the agent is *given* the grid layout (which cells are obstacles, where the goal is, that moves are deterministic), it can plan the shortest safe path directly, with no trial and error
- This is exactly the search problem from Lecture 4: BFS or Dijkstra on the grid graph would find the optimal path immediately
- Model-based RL is, in this sense, “search plus a model you also had to learn,” when the model isn’t handed to you for free

GridWorld, model-free style

- If the agent does *not* know in advance which cells are obstacles, it must actually step into a few bad cells (or watch others do so) to find out
- It must estimate values or a policy purely from the rewards it happens to observe by acting
- This is closer to how RL is usually used in practice: real environments (robots, dialogue, games) rarely hand you an exact transition model

Why this course emphasizes model-free methods

- Real state spaces are often enormous or continuous: pixels, robot joint angles, dialogue histories
- Learning an accurate model of *how the world changes* in response to actions in these spaces is itself extremely hard
- Model-free methods sidestep this by learning directly what to *do*, without first learning how the world *works*
- Trade-off: typically less sample-efficient than model-based planning when a good model is available

The Bellman equation

The recursive structure that makes value functions computable.

- Value of a state = immediate reward + discounted value of what comes next
- In symbols:

$$V^{\pi}(s) = \mathbb{E}\left[r + \gamma V^{\pi}(s') \mid s, \pi \right]$$

- This is a **recursive definition**: V^{π} appears on both sides, one step apart in time

Reading the Bellman equation term by term

- $\mathbb{E}[\,\cdot\,]$: an expectation, because the action (if π is stochastic) and the transition (if P is stochastic) may both be random
- r: the immediate reward for this one step
- $\gamma V^{\pi}(s')$: the discounted value of *everything that happens after*, already summarized in a single number
- The equation says: you never need to unroll the whole future to know a state's value — just one step of reward, plus the value of the state you land in

Example: one Bellman backup on GridWorld

Take cell (2,3) in GridWorld, one step from the goal via “down”, under a policy that always moves toward the goal, with $\gamma = 0.9$:

- From (2,3), action “down” leads to (3,3), the goal, with reward +10, episode ends so $V(\text{goal}) = 0$
- $V^{\pi}(2,3) = r + \gamma V^{\pi}(3,3) = 10 + 0.9 \times 0 = \mathbf{10}$
- Now consider (1,3): action “down” leads to (2,3), reward –1
- $V^{\pi}(1,3) = r + \gamma V^{\pi}(2,3) = -1 + 0.9 \times 10 = \mathbf{8}$
- Values propagate backward from the goal, one Bellman backup at a time — exactly like relaxing edges in Dijkstra

The Bellman equation for Q

- In terms of V^π:

$$Q^\pi(s,a) = \mathbb{E}\big[\, r + \gamma V^\pi(s') \mid s, a \,\big]$$

- Equivalently,

$$Q^\pi(s,a) = \mathbb{E}\Big[\, r + \gamma \sum_{a'} \pi(a' \mid s')\, Q^\pi(s',a') \;\Big|\; s, a \,\Big]$$

- V and Q satisfy the *same* recursive idea; Q simply fixes the first action rather than averaging over it
- This is the object most "value-based" RL algorithms (briefly mentioned in Section 2) try to learn directly

Connection to Lecture 4's search algorithms

- Same recursive idea as **Dijkstra/A***: optimal value decomposes into one step plus the optimal value of the rest
- Dijkstra: $\mathrm{dist}(s) = \min_{s'} \left[\mathrm{cost}(s, s') + \mathrm{dist}(s') \right]$, where dist is the shortest distance to the goal
- Bellman equation: $V(s) = \mathbb{E}\left[r + \gamma V(s') \right]$
- Both are instances of **dynamic programming**: solve small subproblems once, reuse their solutions for bigger ones

The key difference from search

- Search (Lecture 4) assumed a **known**, deterministic graph: edges and costs given upfront
- RL assumes we must often **learn** the graph's rewards and transitions from experience, and they may be stochastic

Search assumed a known graph; RL assumes we must learn the graph's rewards from experience.

This single difference is why RL needs an entirely new algorithmic toolkit, even though the underlying recursive structure is the same.

Solving the Bellman equation exactly: value iteration

When P and R are fully known (the model-based case), we can compute V^* exactly.

1. Initialize $V(s) = 0$ for all states
2. Repeatedly sweep every state: $V(s) \leftarrow \max_a \mathbb{E}[\, r + \gamma V(s') \mid s, a \,]$
3. Stop when V stops changing (converges)
4. Recover the optimal policy: at each state, pick the action achieving that max

Policy iteration (brief mention)

- Alternate between two steps: **evaluate** the current policy's V^{π} exactly, then **improve** the policy by acting greedily with respect to that V^{π}
- Repeat evaluate → improve → evaluate → improve until the policy stops changing
- Converges to the optimal policy, often in fewer sweeps than value iteration, at the cost of a full evaluation each round
- Both value iteration and policy iteration require a **known model** — they are model-based methods

Why exact tabular methods break down

- Value iteration and policy iteration require looping over *every* state (and, for the max, every action) on every sweep
- A robot with continuous joint angles, or an agent seeing raw pixels, has effectively infinitely many states — there is no table to fill in
- This is the same **curse of dimensionality** idea from Lecture 3 (PCA), now hitting the state space instead of the feature space
- Real problems need a way to generalize across states, not enumerate them

Function approximation and sampling

- **Function approximation**: represent V, Q, or π with a parameterized function (e.g. a neural network) instead of a table, so nearby states share information
- **Sampling**: instead of sweeping every state exhaustively, learn from the states the agent actually visits by acting
- Together, these let RL scale to huge or continuous state spaces — at the cost of needing careful, stable learning algorithms

This motivates the rest of today's lecture: **policy gradient methods**, which parameterize the policy directly and learn from sampled trajectories.

Check yourself: Section 2

- In your own words, what does ε-greedy trade off, and what does the ε control?
- What does a model-based agent need that a model-free agent does not?
- Write the Bellman equation for V^{π} from memory, and explain each term
- Why can’t value iteration or policy iteration be run directly on a state space of raw camera pixels?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Policy gradient methods

Parameterize the policy directly

The policy gradient idea

Instead of estimating value functions, directly search over policies.

- Parameterize the policy as $\pi_\theta(a \mid s)$, e.g. a neural network with weights θ
- Objective: $J(\theta)$ = expected return under π_θ
- Goal: adjust θ via gradient ascent, $\theta \leftarrow \theta + \eta \nabla_\theta J(\theta)$
- Directly optimizes what we care about; naturally handles continuous and large action spaces

Parameterizing a policy

- **Tabular softmax policy**: one preference score per (state, action) pair; $\pi(a \mid s) = \exp(\text{score}(s, a)) / \sum_{a'} \exp(\text{score}(s, a'))$
- **Neural network policy**: a network takes state s as input and outputs action probabilities (discrete actions) or a distribution's parameters (continuous actions)
- θ denotes *all* the tunable numbers inside this function — table entries, or network weights
- In GridWorld, θ could be a 16-cell × 4-action table of preference scores, one per (cell, action) pair

The objective, written out formally

- $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[\, G(\tau) \,]$, where $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ is a full trajectory
- In words: sample many trajectories by running the current policy, and average their returns
- We want to find θ that makes this average as large as possible
- The expectation is over *both* the policy's randomness and the environment's randomness

What can be backpropagated, and what cannot

Backprop needs an unbroken chain of known derivatives from θ to the objective.

Piece	To θ?	
Policy probability $\pi_\theta(a \mid s)$	**Yes**	a network we wrote; every derivative is known
Sampled action $a \sim \pi_\theta$	**No**	drawing a sample has no gradient
Reward r and next state s'	**No**	environment is a black box; $\partial s' / \partial \theta$ is unknown
Return G	**No**	reaches θ only through the two rows above

What can be estimated by sampling, and what cannot

The policy is differentiable. The trouble is that the gradient of an expectation is not itself an expectation.

Quantity	Sample?	
$\nabla_\theta J = \sum_\tau \nabla_\theta P_\theta(\tau)\, G(\tau)$	**No**	$\nabla_\theta P_\theta$ can be negative and does not sum to 1; no samples come from it
$J(\theta) = \sum_\tau P_\theta(\tau)\, G(\tau)$	**Yes**	weights P_θ are a distribution: run the policy, average G

We need to rewrite $\nabla_\theta J$ so that a probability weight is back in front.

The score function (“log-derivative”) trick

One identity puts the probability weight back.

- Since $\nabla_\theta \log P = \nabla_\theta P / P$:

$$\nabla_\theta P_\theta(\tau) = P_\theta(\tau)\, \nabla_\theta \log P_\theta(\tau)$$

- Substitute into the gradient:

$$\nabla_\theta J = \sum_\tau P_\theta(\tau)\, \nabla_\theta \log P_\theta(\tau)\, G(\tau) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\nabla_\theta \log P_\theta(\tau)\, G(\tau)\right]$$

- This is an expectation under our own policy, so sampled trajectories estimate it

Sample the return, backpropagate the log-probability

After the trick, each piece has a clear job.

Piece	Handled by	Role
Trajectories and returns G	**Sampling**	no gradient; act as fixed weights
$\nabla_\theta \log \pi_\theta(a_t \mid s_t)$	**Backprop**	through the policy network only
Environment dynamics	**Drop out**	$\nabla_\theta \log P_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$

Positive G makes a_t more likely next time; very negative G makes it less likely.

The REINFORCE algorithm

The simplest policy gradient algorithm, built directly on the theorem.

1. Sample a trajectory by running the current policy π_θ in the environment
2. Compute the return G_t from each time step t onward
3. Update:

$$\theta \leftarrow \theta + \eta \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t$$

4. Repeat with the updated policy

Example: one REINFORCE update

Treat a single bandit pull as a length-1 "trajectory," with 2 actions (arms) and a tabular softmax policy with preference scores $\theta_A = 0, \theta_B = 0$ (so $\pi(A) = \pi(B) = 0.5$ initially).

- Sample action A (since π is 50/50), observe reward $G = +1$
- $\nabla_\theta \log \pi(A)$ for a 2-action softmax at 50/50 works out to ($+0.5$ for θ_A, -0.5 for θ_B)
- Update with learning rate $\eta = 0.1$: $\theta_A \leftarrow 0 + 0.1 \times 0.5 \times 1 = \mathbf{0.05}$; $\theta_B \leftarrow 0 + 0.1 \times (-0.5) \times 1 = \mathbf{-0.05}$
- $\pi(A)$ is now slightly above 0.5 — the policy nudged toward the action that just paid off

Example, continued

- If instead the sampled action had been A but the observed reward were $G = -1$ (a bad outcome), the same formula gives $\theta_A \leftarrow -0.05, \theta_B \leftarrow +0.05$
- The policy nudges *away* from an action that led to a bad outcome, even though it was the action actually taken

This is exactly the “trial and error, made precise as gradient ascent” idea — good outcomes get reinforced, bad outcomes get suppressed, and the size of the nudge scales with how good or bad G was.

Reading the REINFORCE update

- $\log \pi_\theta(a_t \mid s_t)$: how to nudge θ to make action a_t more or less likely
- Multiplying by G_t: scale that nudge by how good the outcome actually was
- If G_t is high, push probability of a_t up; if G_t is low (or negative), push it down
- This is a form of **trial and error**, made precise as gradient ascent

REINFORCE is on-policy

- The trajectory used to compute the update must be sampled from the *current* π_θ, not an old or different policy
- This means every update requires fresh interaction with the environment — old trajectories from a previous θ cannot be reused without correction
- This is one reason policy gradient methods can be sample-inefficient: each batch of experience is “spent” after one (or a few) updates

Why REINFORCE works, and where it struggles

- **Works**: unbiased estimate of the true policy gradient, given enough samples
- **Struggles**: return G_t is a single noisy Monte Carlo sample of a long, stochastic future
- One lucky or unlucky trajectory can swing the gradient estimate wildly
- Result: high variance, slow and unstable learning, many samples needed

A thought experiment: how noisy is the return?

- Suppose an environment where a good policy succeeds 90% of the time but fails badly 10% of the time
- Nine times out of ten, G_t looks great and reinforces the current behavior
- One time out of ten, an unlucky episode produces a large negative G_t for the *exact same* good policy, pushing θ in the opposite direction
- Average over enough episodes and this washes out (unbiased) — but with small batches, a single unlucky trajectory can dominate the update

An unbiased estimator with huge variance is still, in practice, a bad estimator to update on.

Check yourself: Section 3

- What algebraic trick turns “gradient of a probability” into something we can estimate from samples?
- Write the REINFORCE update rule from memory
- Why is REINFORCE described as *unbiased but high-variance*?
- Why must the trajectory used for an update come from the current policy, not an older one?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Variance reduction in REINFORCE

Baselines · Reward-to-go

Why the naive REINFORCE gradient is noisy

- G_t is a sum over an entire (possibly long) future trajectory
- Every reward along the way, and every future action's randomness, contributes noise
- The absolute scale of G_t also varies a lot across environments and episodes, which distorts the gradient magnitude
- We want an estimator that keeps the same expectation (still unbiased) but has smaller variance

Baselines: subtracting expected return

- Replace G_t with $G_t - b$, where b is an estimate of the expected return from that state
- Updated rule:

$$\theta \leftarrow \theta + \eta \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(G_t - b)$$

- Intuition: what matters is whether an action did *better or worse than expected*, not its absolute return
- Common baseline: a learned estimate of $V(s_t)$

Why subtracting a baseline stays unbiased

- Claim: $\mathbb{E}_a[\,\nabla_\theta \log \pi_\theta(a \mid s)\,] = 0$ for any fixed s, when the expectation is taken over actions sampled from $\pi_\theta(\cdot \mid s)$
- Sketch: $\sum_a \pi_\theta(a \mid s)\,\nabla_\theta \log \pi_\theta(a \mid s) = \sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0$, since probabilities always sum to 1
- Because b does not depend on the sampled action, $\mathbb{E}_a[\,\nabla_\theta \log \pi_\theta(a \mid s) \cdot b\,] = b \cdot \mathbb{E}_a[\,\nabla_\theta \log \pi_\theta(a \mid s)\,] = b \cdot 0 = 0$
- So subtracting b changes nothing *in expectation* — it only changes the variance of individual samples

Example: adding a baseline

Return to the 2-action bandit example, $\theta_A = \theta_B = 0$, $\pi(A) = \pi(B) = 0.5$, sampled action A, reward $G = +1$. Suppose the baseline is $b = 0.5$ (a running average of past rewards).

- Without a baseline: update used $G = +1$, giving $\theta_A \leftarrow +0.05$, $\theta_B \leftarrow -0.05$ (as before)
- With the baseline: update uses $G - b = 0.5$, giving $\theta_A \leftarrow +0.025$, $\theta_B \leftarrow -0.025$ – a *smaller*, less noisy step in the same direction
- If the observed reward is *exactly average* ($G = b$), the update becomes zero: "this outcome was exactly as expected, no need to move θ"

Choosing the baseline

- Simplest choice: a running average of observed returns across recent episodes (a single scalar, ignores state)
- Better choice: a *learned* estimate of $V(s_t)$, so the baseline adapts to how good outcomes typically are *starting from this particular state*
- A state-dependent baseline is strictly more informative: "better than typical for this state" is a sharper signal than "better than the global average"
- This learned $V(s)$ baseline is the seed of the **critic**, introduced properly next week

Reward-to-go: a second variance-reduction trick

- An action a_t cannot possibly have caused any reward that occurred *before* time t
- **Reward-to-go**: credit an action only with the rewards from t onward (exactly what G_t already denotes when correctly indexed)
- If an implementation accidentally used the *total episode return* for every single time step's update, it would be crediting early actions for rewards that happened before they were even taken
- Reward-to-go fixes exactly this bug, and is standard practice in every real policy-gradient implementation

Example: reward-to-go on GridWorld

Using the 6-step GridWorld trajectory (rewards −1, −1, −1, −1, −1, +10):

- Reward-to-go from $t = 0$: the full return $G_0 \approx 1.82$ (computed earlier)
- Reward-to-go from $t = 4$ (only two steps remain): $G_4 = r_4 + \gamma r_5 = -1 + 0.9(10) = \mathbf{8}$
- Using the *total* episode return (≈ 1.82) for the update at every single time step would understate how good the last couple of actions were, and overstate how bad the first couple were
- Reward-to-go assigns credit more precisely, step by step

Reward and return normalization

- Normalize returns within a batch of trajectories (subtract mean, divide by standard deviation)
- Keeps gradient magnitudes on a consistent scale across environments and training stages
- A simple, practical variance-reduction trick used alongside baselines and reward-to-go
- None of these three tricks change *what* the algorithm is optimizing — only how noisy each individual update is

Putting it together: the practical recipe

1. Collect a batch of trajectories using the current π_θ
2. For each time step, compute the reward-to-go (not the full-episode return)
3. Subtract a baseline (running average, or learned $V(s)$) from each reward-to-go
4. Optionally normalize the resulting advantages across the batch
5. Take a gradient ascent step using $\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot \text{advantage}_t$

From REINFORCE toward actor-critic

- REINFORCE with a learned baseline is a natural bridge to a more efficient family of methods
- Instead of a Monte Carlo return, use a learned value function to estimate “how good” an action was, updated online, step by step, rather than waiting for a full episode to finish
- The **actor** (policy) and **critic** (value function) are trained together
- This is next week’s topic: Reinforcement Learning 2 — actor-critic methods

Today’s baseline is next week’s critic, just estimated differently.

Check yourself: Section 4

- Why does subtracting a state-dependent baseline not introduce bias into the gradient estimate?
- What specifically does “reward-to-go” fix, compared to using the full-episode return at every time step?
- Name three variance-reduction tricks covered today, in one phrase each
- In your own words, what will an actor-critic method replace, relative to REINFORCE with a baseline?



STEVENS
INSTITUTE OF TECHNOLOGY
1870

From game moves to next tokens

Where else policy gradients show up

RL's toolbox is domain-agnostic

- Nothing about the policy gradient theorem, REINFORCE, or baselines assumed the actions were robot motions or board-game moves
- All that was required: a policy π_θ over some action space, a way to sample trajectories, and a scalar reward signal
- This generality is exactly why the same mathematics reappears in language model post-training (Lecture 7)

Revisiting Lecture 7 through today's lens: PPO

- In RLHF, the "state" is the prompt plus tokens generated so far; the "action" is the next token; the "reward" comes from a learned reward model (or a KL penalty term)
- PPO is a policy-gradient-family method: it still follows the $\nabla \log \pi \cdot (\text{something like an advantage})$ recipe
- What PPO adds beyond vanilla REINFORCE: a *learned critic* (much like today's baseline discussion) and a *clipped update* that prevents the policy from moving too far in one step
- We will see the clipping mechanism properly in Lecture 10, once actor-critic machinery is in place

Revisiting Lecture 7 through today's lens: GRPO

- GRPO also follows the $\nabla \log \pi \cdot (\text{advantage})$ recipe, but estimates the advantage differently: by comparing a *group* of sampled completions for the same prompt against each other
- This is conceptually close to today's baseline idea — the group average plays the role of b , the baseline being subtracted off
- GRPO drops the separate learned critic network that PPO needs, trading a bit of estimator quality for a much simpler, cheaper training loop
- Verifiable rewards (a math checker, a unit test) replace the learned reward model wherever the task allows it

Side-by-side: REINFORCE, PPO, GRPO

Method	Reward source	Baseline / critic
REINFORCE	environment reward	none, or running average
PPO	learned reward model	learned critic (V)
GRPO	verifiable or learned reward	group-relative average, no critic

Update style: REINFORCE takes one unclipped step; PPO and GRPO both **clip** the update so the policy cannot move too far from the data it was sampled from.

Why this table matters for the rest of the course

- Every method in this row is answering the *same* underlying question: “which of my recent actions should I make more or less likely, and by how much?”
- The differences are all about *where the advantage/baseline comes from* and *how aggressively to trust one batch of data*
- Once you understand REINFORCE and baselines deeply, PPO and GRPO are refinements, not a different subject
- This is also why Lecture 10 (actor-critic) is really “REINFORCE’s baseline, made more sophisticated,” not a brand-new topic

Any environment, any reward

- A dialogue agent: state = conversation so far, action = next utterance, reward = user satisfaction or a scoring model
- A trading agent: state = market features, action = buy/sell/hold, reward = profit and loss
- A recommendation system: state = user history, action = which item to show, reward = click or purchase
- The pattern is always: define states, actions, and a reward signal, then reach for this same toolbox

Check yourself: Section 5

- In one sentence, what do PPO and GRPO have in common with REINFORCE?
- What role does the “group” play in GRPO, compared to the baseline b in today’s REINFORCE discussion?
- Name one domain outside games or robotics where you could frame a problem as an MDP



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Recap and looking ahead

MDPs · Policy gradients · What’s next

Recap: the MDP formalism

- States, actions, transitions, rewards, and a discount factor define the problem
- The Markov property is what lets us summarize “everything relevant” in a state, without remembering the full history
- Policies map states to actions (or action distributions); value functions score states and state-action pairs
- The Bellman equation gives value functions their recursive structure, closely paralleling the dynamic-programming idea behind Dijkstra and A* from Lecture 4

Recap: policy gradients and REINFORCE

- Policy gradients parameterize the policy directly and climb the gradient of expected return
- The score-function trick, $\nabla \log \pi$, is what makes this gradient estimable from sampled trajectories alone, with no model of the environment
- REINFORCE is the simplest instance: sample a trajectory, compute returns, take a weighted gradient step
- It is unbiased but high-variance — exactly the problem the rest of this unit exists to fix

Recap: the variance-reduction toolkit

- **Baselines**: subtract an expected-return estimate; unbiased because $\mathbb{E}[\nabla \log \pi] = 0$ for a fixed state
- **Reward-to-go**: only credit an action with rewards that happen *after* it, not before
- **Normalization**: rescale returns within a batch for a consistent gradient magnitude
- Together, these turn “REINFORCE” from a textbook idea into something that actually trains in a reasonable number of samples

Looking ahead: actor-critic methods

- Next week (Lecture 10) replaces the Monte Carlo return with a *learned critic*, updated continuously rather than only at the end of an episode
- The **advantage function**, $A(s, a) = Q(s, a) - V(s)$, becomes the natural generalization of "return minus baseline"
- PPO's clipped update and generalized advantage estimation (GAE) will connect directly back to Lecture 7's RLHF discussion
- Everything from today — the Bellman equation, baselines, reward-to-go — is a direct prerequisite for next week's material

Closing thought

Today's story: cast decision-making as an MDP, then learn a policy directly from sampled trajectories.

- MDP formalism: states, actions, transitions, rewards, discount, and the Bellman recursion
- Model-free policy gradients: REINFORCE increases the probability of actions that led to high return
- REINFORCE is unbiased but high-variance; baselines, reward-to-go, and normalization are the fixes
- **Connection to Lecture 7**: RLHF's PPO and RLVR's GRPO are also policy-gradient-family methods — same core idea ($\nabla \log \pi \cdot \text{advantage}$), applied to language model outputs instead of game or robot actions

Policy gradients are the same mathematical object whether the "actions" are game moves or next-token choices.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

THANK **YOU**

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030